CHAPTER 9: CHUNKING, INDEXING, AND THE QUERY YOU DID NOT ANTICIPATE

Summary: *Nearly every retrieval pipeline gets tuned in the wrong place. Teams adjust how much comes back, how often the index rebuilds, and how the answer is generated, when the defect is what a single retrieved piece contains. This chapter separates the decisions that change the content of a chunk from the ones that only change its quantity, sets out where stale material actually enters and leaves a pipeline, and covers the documented costs of feeding documents to a model.*

Chunk Size Is Not the Chunking Decision

The characteristic failure is easy to describe and hard to see in an option list. A retrieved passage contains part of an answer, and the part that governs it lives somewhere else: the exception is in a different clause, the definition is in a different section, the qualifier is two paragraphs up.

Now watch the remedies people reach for. Increase the number of passages retrieved. Enlarge the chunks. Add overlap between adjacent chunks. Retrieve, then re-rank.

Every one of those changes how **much you get back**. None of them changes what a **retrieved piece contains**. More passages means more of the same kind of fragment. Bigger chunks means a bigger arbitrary window that still cuts wherever it happens to land. Overlap means the boundary moves slightly, which helps only if the governing text happened to sit just outside it.

The fix is to **cut each retrieval unit** so that it **stands on its own as a piece of meaning**, with **whatever it leans on attached** to it. A clause carrying its own definitions and exceptions can answer a question by itself. Four hundred tokens of whatever happened to fall inside a window cannot, and no human reading it would think otherwise.

Name it: **chunk by corpus, index by query**. Chunk boundaries follow the structure of the material you have. What you index and how you match follows the questions people ask. Those are different decisions, and they are usually made as one, badly.

The Citable Unit Is the Block You Supplied

If a scenario cares about attribution granularity, one documented fact settles it.

A **text block** is the **minimal citable unit**, and Claude cites **whole blocks** rather than **substrings** inside them. Finer-grained citations are obtained by splitting the content into smaller blocks.[1]

That converts a presentation complaint into a chunking decision. When somebody says the citations are too coarse and point at a whole page, no prompt instruction improves it. Your **block boundaries are your citation boundaries**.

Two related details are worth having exactly.[1]

Citations are disabled by default for search results. A design that assumes attribution is automatic will ship without it.

When a search **fails or returns nothing**, return a **plain text block describing what happened** rather than **raising an error**. The model then explains the empty result and the conversation continues. Raising an error converts a normal outcome, no matching documents, into a broken turn.

The document types differ in how they are chunked and cited, and the third row is the one that matters for control:[2]

Document type	How it is chunked	How it is cited
Plain text	Automatically chunked into sentences	By character indices
PDF with text content	Automatically chunked into sentences	By page number
Custom content	**No additional chunking**	By **block indices**, and recommended where you want more granular citations

Custom content is the option that gives you the boundaries, because nothing re-chunks it after you decide. If a scenario wants control over what gets cited, that is the row.

What a PDF Actually Costs

Feeding documents in has a documented price, and the range is wide enough to change a design.[3]

A **page of PDF typically consumes 1,500 to 3,000 tokens** depending on how dense it is. And the processing mode matters more than most people expect: text extraction alone uses roughly **1,000 tokens for a 3-page PDF**, while full visual understanding, which processes every page as both text and image, uses roughly **7,000 tokens for the same 3 pages**.

That is a sevenfold difference for the same document, so the **mode is a cost decision, not a quality preference**. Visual understanding earns its price on documents where layout carries meaning, tables, forms, diagrams, signatures. On a wall of paragraphs it is paying seven times over for information the text already contains.

One more consequence follows from the same page: PDF support relies on Claude's vision capabilities, so it **inherits the considerations that apply to any vision task**. A design treating PDF ingestion as a text problem has missed that.

Stale Content Enters at Ingestion and Leaves at Filtering

When a pipeline keeps returning out-of-date material, there are exactly two stages where you can act on it, and the reason is structural rather than a matter of preference.

Stale content enters at ingestion. Something wrote it into the index.

Stale content leaves at retrieval filtering. Something declined to return it.

Every other stage sits downstream of the set of passages. Retrieving more of them, rebuilding more often, or changing how the answer is generated all operate on or

after a passage set that already contains the stale material. You **cannot fix the contents of a set** by adjusting what **happens to it afterwards**.

That is why, when a scenario asks for two measures against staleness, the pair is ingestion and filtering. Cadence sounds like the obvious answer and is not: rebuilding nightly still leaves a full day of staleness, and if the problem is that superseded documents were never removed, no rebuild frequency touches it at all.

Embeddings, and the Side the Text Sits On

Two facts that scenarios are built on.

Anthropic does not offer its own embedding model and points to a third-party provider.[4] An option that describes calling an Anthropic embedding endpoint is describing something that does not exist.

Documents and queries are embedded with different input types, marking which side of the retrieval each piece of text is on.[4] That asymmetry is easy to lose in an implementation and produces quietly degraded matching rather than an error.

There is a parallel worth drawing to the previous chapter's tool-catalogue problem, because the documented solution has the same shape. The tool search tool keeps a very large catalogue usable by searching it and loading only the definitions a task needs, rather than loading everything up front. Two variants exist, one where Claude builds regular expression patterns and one where it issues natural language queries against BM25, and a custom implementation can use embeddings or semantic search.[5] **Selecting from a large set at request time** is the **same problem** whether the set is **documents or tools**, and BM25 appearing beside embeddings here is the same lexical-versus-semantic split from chapter 6. Deferral is not a universal improvement, and the documentation says where it stops paying: **standard tool calling fits better under about ten tools**, when **every tool is used on every request**, or when the definitions are **small enough not to matter**. Deferred definitions still **cost input tokens once loaded**, so what changes is when they are **charged**, not whether.

How Wrong Answers Are Built on Retrieval Pipeline Questions

- **Quantity offered against a content defect**. The option retrieves more passages, enlarges chunks, adds overlap or re-ranks. The test: does this change what is **inside a retrieved piece**, or only how **many pieces** there are?
- A **downstream stage offered against an upstream problem**. The option tunes generation, breadth or cadence against material that is already in the passage set. The test: at which **stage does the bad content enter**, and at which **stage can it be excluded**?
- **Prompting offered against a structural boundary**. The option asks for finer citations by instruction. The test: what is the **smallest unit the system can cite**, and who **decided that boundary**?
- **Assuming a default that is off**. The test: does this **behaviour need enabling**? Citations for search results do.

When the scenario says	The answer is usually	The trap is usually
Retrieved text misses the governing clause, definition or exception	Cut units that stand alone, and attach what they lean on	More passages, bigger chunks, overlap, or re-ranking
Citations are too coarse to be useful	Split the content into smaller blocks	Instructing the model to cite more precisely
You need control over citation boundaries	Custom content, which is not re-chunked and is cited by block index	Plain text or PDF, both auto-chunked into sentences
Attribution is expected on search results	Enable it; it is off by default	Assuming citations are automatic

When the scenario says	The answer is usually	The trap is usually
A search returns nothing, or fails	Return a plain text block describing the outcome	Raising an error, which breaks the turn over a normal result
Stale material keeps being returned, two measures wanted	Act at ingestion and at retrieval filtering	Rebuild cadence, broader retrieval, or generation changes
PDF cost is a concern	Choose the processing mode deliberately; visual understanding costs about seven times text extraction	Treating mode as a quality setting with no price
Layout carries meaning: tables, forms, diagrams	Full visual understanding earns its cost	Text extraction, which discards the thing that mattered
An embedding model is needed	A third-party provider; Anthropic does not offer one	An option describing an Anthropic embeddings endpoint
A very large tool catalogue is hurting the context	Search the catalogue and load only what the task needs	Loading every definition up front

The habit worth carrying: for any proposed retrieval change, ask whether it alters the **contents of the passage set** or only its **size**. Most wrong answers here are quantity changes offered against a quality problem.

What to Remember

- Retrieval failures where the **governing text is missing** are **content defects, not quantity defects**. More passages, larger chunks, overlap and re-ranking

all change how **much comes back** and none of them changes what a **retrieved piece contains**.

- The fix is a retrieval unit that **stands on its own as a piece of meaning**, with **whatever it leans on attached**: a clause carrying its own definitions and exceptions, rather than a **fixed window that cuts wherever it lands**.
- Chunk by **corpus**, index by **query**. Boundaries follow the **structure of the material**; matching strategy follows the **questions people actually ask**. These are **two decisions** and they **get made as one**.
- A text block is the **minimal citable unit** and **whole blocks get cited, never substrings**, so finer citations come from **smaller blocks**, not from instructions. Your **block boundaries** are your **citation boundaries**.
- **Citations are off by default** for search results. When a search fails or returns nothing, **return a plain text block describing the outcome** so the model can explain it, rather than **raising an error and breaking the turn**.
- **Plain text** is auto-chunked into sentences and cited by **character index**; **PDFs with text** are auto-chunked into sentences and cited by **page**; **custom content** receives **no additional chunking** and is cited by **block index**, which makes custom content the choice when you want to **control granularity**.
- A PDF page typically costs **1,500 to 3,000 tokens**. Text extraction alone runs about **1,000 tokens for 3 pages** against roughly **7,000 tokens for the same 3 pages** with full visual understanding, so the **mode is a cost decision**, worth paying only where **layout carries meaning**. PDF support runs on **vision capabilities** and inherits their considerations.
- Stale content enters at **ingestion** and can be excluded at **retrieval filtering**. Those are the **only two stages** that act on it. Rebuild cadence, retrieval breadth and generation are all **downstream of the passage set** and cannot change what is in it.

- **Anthropic does not provide an embedding model**, and documents and queries are embedded with **different input types** marking which side of the retrieval each sits on.
- A very large tool catalogue is solved the **same way as a large corpus**: **search it at request time** and **load only what is needed**, by regular expression, by natural language against BM25, or by a custom embedding-based search.